

PAPER_507: Wolfram Field Unity Engine — Hypergraph Spacetime Evolution

Unified Quantum Field Framework (UQFF)

Daniel T. Murphy • UQFF Research Consortium • 2026

Source: PAPER_507_Wolfram_Field_Unity_Engine_Hypergraph.md

PAPER_507: Wolfram Field Unity Engine — Hypergraph Spacetime Evolution

Session: 137 | **Source:** grok_share_84a767d3.txt (lines 3800–4200) **Date:** December 2025 — commit df7e222 (final WSTP integration) **Related files:** source177_wolfram_field_unity.cpp (WolframFieldUnityEngine class)

§1.1 Abstract

The WolframFieldUnityEngine implements a discrete spacetime model grounded in the Wolfram Physics Project hypergraph formalism. A multiway graph $G \subset \{(node\blacksquare, node\blacksquare, node\blacksquare)\}$ evolves via the [sacredMagneticOrbitRule](#), which introduces new nodes stochastically at each step. Dimensionality is inferred via BFS visit-count power law. Buoyant gravity emerges from edge-flux density relative to total node count — no Newtonian constant G required.

§1.2 Initial Consciousness Seed

The hypergraph is initialized from a void (singleton node 0) plus 26 directed edges, one per UQFF dimension:

```
G_0 = { {0} }                (void node – dimensional origin)
G_1 = { {0,1}, {0,2}, ..., {0,26} } (26 branches from void)
```

Total nodes after seed: 27 (void + 26 dimensional projections)

This maps 1-to-1 to the 26 UQFF field dimensions (Ug1, Ug2, ..., Ug4 per dimension plus quantum state factors).

§1.3 sacredMagneticOrbitRule

At each evolution step, the update rule selects two existing nodes randomly and appends one new node:

```
Rule(G):
  n1 ← uniform_int(0, |nodes|-1)
  n2 ← uniform_int(0, |nodes|-1)
  G ← G ∪ { {n1, n2, ++max_node} }
  max_node ← max_node + 1
```

This is a binary-spawn ternary hyperedge rule:

- Binary selection from existing nodes ensures no isolated creation
 - Ternary edge {n1, n2, new} mimics branching geodesic
 - The random selection produces causal asymmetry (time arrow)
-

§1.4 evolveMultiway(depth)

Multiway parallelism is achieved via OpenMP:

```
evolveMultiway(depth):
  #pragma omp parallel for schedule(dynamic)
  for i = 0 to depth:
    branch_i = clone(current_graph)
    apply sacredMagneticOrbitRule(branch_i)
    apply sacredMagneticOrbitRule(branch_i)      (double application per branch)
    results[i] = branch_i
```

Node count growth rate $\approx 2^{\text{depth}}$ per evolution pass

Expected: exponential topological complexity

§1.5 measureDimension(center, radius)

The Hausdorff-like dimension is estimated via BFS visit count:

$$D(\text{center}, r) = \log(|\text{visited}(\text{BFS}, \text{center}, r)|) / \log(r + 1)$$

Algorithm:

```
queue ← {center}
visited ← {}
Steps:
  while queue not empty and dist < radius:
    pop node n from queue
    for each edge e containing n:
      for each node m in e if m not in visited:
        visited ← visited ∪ {m}
        push m
```

Dimension formula derived from:
|visited| ≈ r^D (for metric spaces)
→ D ≈ log(|visited|) / log(r)

For flat spacetime this gives D ≈ 3 at large radii; for fractal structures D can be non-integer.

§1.6 measureBuoyantGravity(center)

g_B(center) = Σ_{e ∈ G : center ∈ e} 1.0 / max_node

= (edge degree of center node) / (total node count)

This is a dimensionless gravitational proxy:
- Edge degree = local connectivity = local “mass”;
- max_node ≈ total “volume” of the graph
- Ratio = buoyancy density, unit-consistent with UQFF g_B definition

No Newtonian G appears — gravity is purely topological.

§1.7 Comparative: Wolfram Hypergraph vs UQFF Buoyancy

Property	Wolfram Hypergraph	UQFF Buoyancy (MAIN_1)
Gravity source	Edge-degree / node-count	F_Bi / ρ_fluid
Dimensionality	log count / log radius	26 fixed dimensions
Evolution rule	sacredMagneticOrbitRule	Lagrangian ODE integration
Spacetime fabric	Emergent from graph	Pre-declared fluid medium
Quantum state	Max node index	H_SCm, U_UA parameters

§1.8 Graph-Theoretic Equations

Node set: V(G) = {0, 1, ..., max_node}
Edge set: E(G) ⊂ V³ (ternary)
Dimension: D(c,r) = log(|B_r(c)|) / log(r+1)
Gravity: g_B(c) = deg(c) / |V(G)|
Evolution: G_{t+1} = G_t ∪ { {n1(t), n2(t), max_node+1} }
Branching: G^(k)_{t+1} = evolve(G^(k)_t) for k ∈ [0, depth) ■

§1.9 Citation

Source: grok_share_84a767d3.txt, lines 3800–4310 (source177_wolfram_field_unity.cpp) Commit: df7e222 — "Session 129 final: WSTP integration complete" Related: PAPER_506 (PI Infinity Decoder), PAPER_508 (Sacred Time) Implementation: [WolframFieldUnityEngine](#) class, source177_wolfram_field_unity.cpp
Paper number: PAPER_507